

《掀开 AI 程序员的引擎盖：Claude Code 与 AI Agent 的底层架构与控制流》

课程受众：高级研发工程师、架构师、效能团队 课程时长：4 小时 演示模型：Qwen3-30B-A3B（MoE，30B 总参 / 3B 激活，失败模式可控可解释） 演示平台：MVP 代码库（mvp/：model_server + client + parser + tools）

课程双主线：- 任务结构线：Localization → Repair → Validation —— Agent 在做什么 - OODA 演进线：每篇论文优化了决策循环的哪个环节 —— 如何做得更好

时间	主题	内容
14:00 - 14:30	破冰：从“文本接龙”到“闭环控制”	Chat → Agent，质变在哪？ - 从维纳控制论到 Boyd OODA：F-86 vs MiG-15 —— 快速的“足够好”胜过迟到的“完美” - ReAct 论文：同一模型加闭环，准确率从 30% 跃升到 60%+ —— 循环机制是放大器 - LLM vs Harness 分工表：Agent 超半数关键工作由工程代码完成，模型甚至不知情 - 课程双主线与二维认知地图：11 篇论文在 L-R-V × OODA 中的定位 【Demo 1】 同一个 bug 两种修法 —— Chat 只能猜，Agent 自主 Read→Edit→Bash 验证
14:30 - 15:10	工具调用：模型如何“伸出手”	模型怎样从“只会说”变成“能操作”？ Action Space 四流派：SWE-agent ACI（消融实

时间	主题	内容
		验：同模型 7 倍差距）、AutoCodeRover AST、CodeAct 代码即动作、Agentless 固定流水线 → 收敛为两条工业路线 • CC 工具集精简设计：~9 核心 + ToolSearch 延迟加载，Edit search/replace 行业共识 • 只读并行 / 写入串行：CC 工具编排的并发控制策略 • 三家协议对比（Claude / OpenAI / Qwen）+ adapter 转换 + parser 多层 fallback 【Demo 2】 chat vs tool_use 并排 curl 对比；adapter 协议转换链路；parser 三格式鲁棒解析
15:10 - 15:20		课间休息
15:20 - 15:50	修 Bug 实战：多轮纠错与防死循环	测试报错后的那条 Thought，决定了纠错还是死循环 • Orient 是关键时刻：“Expected 70.0, got 87.5” → “350/4, 除数该是 5 不是 4” —— 这条推理决定一切 • Reflexion：用自然语言替代梯度 —— HumanEval 80.1% → 91.0%，不改权重只改记忆 • 上下文压缩：模型给自己写摘要 —— 9 段结构中“Errors and fixes”确保失败经验不丢

时间	主题	内容
		- 死循环三层防线：信息质量保障 → 行为检测纠正 → 强制终止止损 【Demo 3】 现场修 buggy_calc.py 两个 bug —— 第一个修完测试仍失败，观察模型如何 Orient
15:50 - 16:20	ACI 设计：百万行代码中精准定位	如何用“刚好够”的信息量找到目标？ - RAG 在编程场景的局限：自然语言 vs 代码符号的语义鸿沟 - 三种 Localization 策略对决：SWE-agent ACI vs AutoCodeRover AST vs Agentless 三层漏斗 - 信息粒度工程：Read offset/limit、Grep head_limit、文件树 50 行上限 —— 粗细粒度漏斗 - 写入范式：Unified Diff 行号定位的脆弱性 → search/replace 内容定位的行业共识 【Demo 4】 cat 读 200 行文件 vs Read 精准读取 —— 上下文爆炸对推理质量的影响
16:20 - 16:30		课间休息
16:30 - 16:50	架构反思：流水线 vs Agent 循环	修一个 Bug，一定需要完整的 Agent 循环吗？ - Agentless 全流程：Localization 漏斗 → 多候选采样 → 测试筛选，\$0.70 / 32% 解决率 - 广度 vs 深度：并行 10 个候选（无反馈）vs 串行试错（有反馈），两种搜

时间	主题	内容
		索哲学 - 流水线的脆性：定位错则全盘皆输 —— 动态纠偏是 Agent 循环不可替代的核心价值 - 按任务难度匹配：简单问题用流水线，复杂问题用 Agent，80/20 混合最优 【Demo 5 • 图解】 同一个 bug 两条路径并排：Agent 循环 vs Agentless 流水线
16:50 - 17:20	Agent Team：从单体到协同编排	一个 Agent 搞不定的任务，怎么拆？ - 三种场景才值得 Team：上下文污染、并行探索（“彻底性而非加速”）、工具专业化 - 四种架构对比： MetaGPT 固定 SOP / AutoGen 对话驱动 $O(N^2)$ / OpenHands 硬编码 / CC 原语工具化 - CC 星型拓扑：Worker 不互通只向 Lead 汇报，$O(N)$ 协调 + Git Worktree 物理隔离 - 三层模型金字塔：Opus 做规划、Sonnet 做执行、Haiku 做杂活 —— 编排用强模型，执行用弱模型 【Demo 6】 Todo 服务多 Agent 协作 —— Lead 拆解 → Worker 并发开发 → 端口 mismatch 被 Meta-V 发现

时间	主题	内容
17:20 - 17:30	安全与权限：Agent 的行为边界	能力越大，边界越重要 • 三道防线：工具级权限（7 种模式）→ 内容级规则（8 来源层级覆盖）→ 执行约束（超时 + 拦截） • Hook ≠ Tool：模型不可见的 harness 层事件触发器，自动化质量门禁 • CLAUDE.md 5 层加载：自然语言“宪法”—— 结构化工具集是精细权限控制的前提
17:30+	〔加餐〕Meta-Harness + 课程总结	Agent 优化 Agent —— 课程的终点恰好是下一个起点 • 同一模型换 harness 可差 6 倍：Meta-Harness 用 coding agent 自动搜索更好的 harness • 关键发现：只给分数 → 34.6，给完整轨迹 → 50.0 —— 诊断细节不可丢 • TerminalBench-2：Haiku 上 #1 击败所有手工方案，Opus 上 #2 超手工设计冠军 • 四句总结：循环转速决定胜负、Harness 工程决定上限、协同架构决定规模、安全机制决定底线（选讲，视时间决定展开深度）

授课建议

1. 以 **Boyd 故事破冰**：F-86 vs MiG-15 的空战案例开场，从军事决策理论自然过渡到 ReAct，建立课程理论纵深。

2. **认知地图常驻**：L-R-V × OODA 的论文定位图打印或常驻屏幕一角，每讲完一节标记当前位置。
3. **先展示失败**：先演示模型在该环节的典型错误（幻觉、死循环、上下文丢失），再讲工程应对，建立可信度。
4. **API 对比是核心教具**：`chat completion` vs `tool_use` 并排 curl，提前准备可运行示例。
5. **失败即教材**：小模型 Demo 出错时不要重启，当场标注“OODA 哪个环节失灵”——这是选小模型而非前沿模型的核心原因。
6. **课前预热**：模型加载需数分钟，14:00 前完成 `model_server.py` 启动和 GPU 预热。
7. **CC 源码实证**：课程中标注“CC 源码实证”的论点均可在开源 CC 代码中找到对应。详细索引见 `lectures/reference_cc_source_analysis.md`，可作课后进阶材料。

v3.7 | 2026-04-07 | 全面精简课程表：删除预录标注（全部改为现场演示），内容列压缩为要点式，突出核心问题与关键数据 v3.6 | 2026-04-07 | Agent Team 新增模型路由（三层金字塔、opusplan、sub-agent inherit）；reference 新增 § 10 模型路由分析 v3.5 | 2026-04-02 | 演示模型更换为 Qwen3-30B-A3B (MoE)，授课时间调整为 14:00-18:00 v3.4 | 2026-04-02 | 安全权限模块 + Meta-Harness 加餐 + 四句总结 v3.3 | 2026-04-02 | Agent Team 重构：Anthropic 官方三场景框架，四种架构对比，Todo 服务 Demo v3.2 | 2026-04-01 | 修 Bug 模块重构：聚焦 Orient, Reflexion 原始数据，6 层压缩策略 v3.1 | 2026-04-01 | 基于 CC 开源代码审查：源码实证全面增强 v3.0 | 2026-03-25 | 新增 Demo 策略，选型 Qwen2.5-Coder-7B